

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

NAME: Mukesh Raj L

REG NO: 192571036

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Mock Interview (Low)

Assessment Tool Weightage: 30%

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5

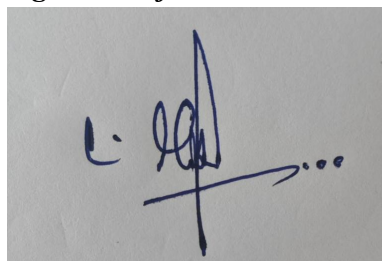
8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

I Mukesh Raj L(192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated

Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

Q1. Explain the Steps of Query Processing in DBMS.

How Does a Query Optimizer Decide the Best Execution Plan?

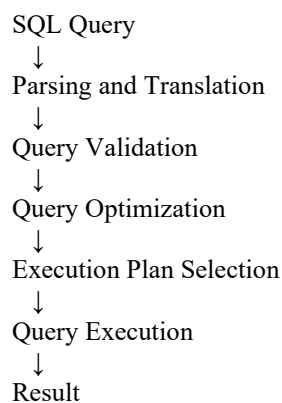
Interviewer Question:

“Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?”

Interview-Style Answer

Query processing is the process by which a DBMS converts an SQL query into an efficient execution plan and produces the required result.

Steps of Query Processing



1. Parsing:

The DBMS checks the SQL syntax and converts the query into an internal representation, such as a query tree.

2. Validation:

It verifies whether the tables, attributes, and operations used in the query are valid.

3. Query Plan Generation:

The DBMS creates possible ways to execute the query.

4. Query Optimization:

The optimizer compares different execution plans and selects an efficient one.

5. Query Execution:

The execution engine performs the operations and retrieves the required data.

How the Optimizer Selects the Best Plan

The optimizer considers factors such as:

- Number of rows in tables
- Available indexes
- Join order
- Estimated I/O cost
- CPU and memory usage
- Size of intermediate results

It estimates the cost of different plans and generally chooses the plan with the **lowest estimated execution cost**.

Example

For a query joining three tables:

STUDENT ⋈ ENROLLMENT ⋈ COURSE

Different join orders may be possible. The optimizer estimates the cost of each order and selects the one that produces fewer intermediate results and requires less processing.

Q2. Difference Between Heuristic Optimization and Cost-Based Optimization

Interviewer Question:

“What is the difference between heuristic optimization and cost-based optimization? When would you use each?”

Interview-Style Answer

Both techniques are used to improve query execution, but they choose an execution plan in different ways.

1. Heuristic Optimization

Heuristic optimization uses **predefined rules** to improve a query plan. It does not mainly compare numerical execution costs.

Common rules include:

- Perform **selection operations early** to reduce rows.
- Perform **projection early** to remove unnecessary columns.
- Execute joins in an order that reduces intermediate results.

Example:

Apply WHERE condition first
↓
Reduce number of rows
↓
Perform JOIN

2. Cost-Based Optimization

Cost-based optimization generates different possible execution plans and **estimates their execution cost**. It may consider:

- Number of rows
- Number of pages
- Available indexes
- Join order
- I/O cost
- Size of intermediate results

The optimizer generally selects the plan with the **lowest estimated cost**.

Example:

Plan 1 → Estimated Cost = 100 I/O
Plan 2 → Estimated Cost = 60 I/O
Plan 3 → Estimated Cost = 80 I/O

Select Plan 2

Difference Between Them

Heuristic Optimization	Cost-Based Optimization
Uses predefined rules	Uses estimated execution costs
Simpler and faster	More detailed analysis
Does not require extensive cost calculation	Compares different plans
Uses rules such as selection pushdown	Uses statistics and cost estimates
Useful for simple optimization	Useful when multiple plans must be compared

When Would You Use Each?

I would use heuristic optimization when a quick and simple optimization is needed. For example, pushing selections down early can immediately reduce the amount of data processed.

I would use cost-based optimization when there are several possible execution plans, especially for complex queries involving multiple joins. The DBMS can compare the estimated costs and choose the most efficient plan.

Q3. Database Connectivity: JDBC vs ODBC

Interviewer Question:

“How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.”

Interview-Style Answer

In a real project, I would implement database connectivity by establishing a connection between the **application and the database**, executing SQL queries, handling transactions, and properly closing or reusing connections.

Basic Steps

Application
↓
Load Database Driver
↓
Establish Connection
↓
Create Statement
↓
Execute SQL Query
↓
Process Result
↓
Commit / Rollback
↓
Close Connection

Example Process

1. **Load the database driver** required for the database.
2. **Establish a connection** using database URL, username, and password.
3. **Create a statement or prepared statement.**
4. **Execute SQL queries** such as SELECT, INSERT, UPDATE, or DELETE.
5. **Process the result** returned by the database.
6. **Commit** the transaction if successful.
7. If an error occurs, perform a **rollback**.
8. **Close or return the connection** after use.

For a large real-world application, I would use **connection pooling** so that connections can be reused efficiently.

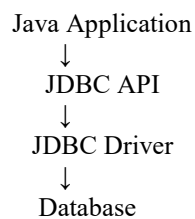
JDBC

JDBC (Java Database Connectivity) is used by **Java applications** to connect and interact with databases.

It provides Java APIs for:

- Connecting to databases
- Executing SQL statements
- Retrieving results
- Managing transactions

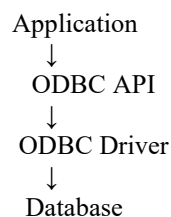
Example flow:



ODBC

ODBC (Open Database Connectivity) is a standard interface that allows applications to access different database systems through an ODBC driver.

Example flow:



Difference Between JDBC and ODBC

JDBC	ODBC
Stands for Java Database Connectivity	Stands for Open Database Connectivity
Mainly designed for Java applications	Can be used by applications written in different languages
Uses JDBC drivers	Uses ODBC drivers
Java-specific API	General database connectivity standard
Works through Java environment	Provides a common interface for database access

Q4. ACID Properties with Real-World Examples

Interviewer Question:

“What are ACID properties? Give a real-world example where each property is critical.”

Interview-Style Answer

ACID properties ensure that database transactions are processed **reliably and consistently**.
ACID stands for:

A → Atomicity
C → Consistency
I → Isolation
D → Durability

1. Atomicity

Atomicity means a transaction is treated as **one complete unit**. Either all operations are completed, or none of them are applied.

Example: Bank Transfer

Suppose ₹1,000 is transferred from Account A to Account B:

Deduct ₹1,000 from A
↓
Add ₹1,000 to B

Both operations must occur. If the system fails after deducting money from A, the transaction must be rolled back.

Critical because: Money should not disappear during a failed transaction.

2. Consistency

Consistency means a transaction takes the database from **one valid state to another valid state**, while following all rules and constraints.

Example: Bank Transfer

Before transfer:

$A + B = \text{Total Balance}$

After the transfer, the total balance should remain correct. Database constraints and rules must not be violated.

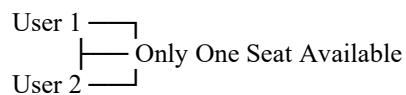
Critical because: Invalid or inconsistent data should not be stored.

3. Isolation

Isolation means multiple transactions should execute without improperly interfering with each other.

Example: Online Ticket Booking

Suppose only one seat is available and two users try to book it simultaneously.



The database must control concurrent access so that the same seat is not booked twice.

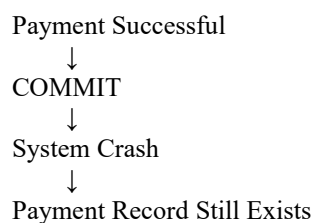
Critical because: It prevents concurrency problems such as double booking.

4. Durability

Durability means that once a transaction is successfully **committed**, its changes remain permanently stored, even if the system crashes.

Example: Online Payment

After a payment is successfully completed:



Critical because: Committed transaction data must not be lost.

ACID Summary

Property	Meaning	Real-World Example
Atomicity	All or nothing	Bank transfer
Consistency	Maintains valid database state	Correct account balances
Isolation	Transactions do not interfere	Online ticket booking

Durability Committed data remains permanent Online payment

Key point to remember:
Atomicity → All or Nothing
Consistency → Valid State
Isolation → No Interference
Durability → Permanent After Commit

Q5. Concurrency Control Problems and How 2PL Solves Them

Interviewer Question:

“Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?”

Interview-Style Answer

Concurrency control is needed when **multiple transactions access the same data at the same time**. Without proper control, problems such as dirty reads, lost updates, and unrepeatable reads can occur.

1. Dirty Read

A **dirty read** happens when one transaction reads data written by another transaction **before it is committed**.

Example:

T1: Update Balance = 5000
T2: Read Balance = 5000
T1: ROLLBACK

Here, T2 has read a value that was later cancelled by T1.

2PL solution: Locks control access to the data. In **Strict 2PL**, an exclusive lock is held until commit or rollback, preventing other transactions from reading uncommitted data.

2. Lost Update

A **lost update** occurs when two transactions update the same data and one update overwrites the other.

Example:

Initial Balance = 1000

T1 reads 1000
T2 reads 1000

T1 updates → 1200
T2 updates → 1500

T1's update may be lost because T2 overwrites it.

2PL solution: An **exclusive lock (X-lock)** ensures that only one transaction can update the data item at a time. The other transaction must wait.

3. Unrepeatable Read

An **unrepeatable read** occurs when a transaction reads the same data twice but gets different values because another transaction modifies the data between the two reads.

Example:

T1: Read Salary = 50,000
T2: Update Salary = 60,000 and COMMIT
T1: Read Salary again = 60,000

T1 receives two different values.

2PL solution: The transaction holds the required lock during its operations, preventing conflicting modifications while the transaction is still using the data.

How 2PL Works

Two-Phase Locking (2PL) has two phases:

Growing Phase
Acquire Locks Only
↓
Shrinking Phase
Release Locks Only

A transaction cannot acquire new locks after it starts releasing locks.

Lock Types

S-Lock → Used for Reading
X-Lock → Used for Writing

The locks ensure that conflicting operations are properly controlled.

Summary

Problem	Meaning	2PL Solution
Dirty Read	Reading uncommitted data	Prevent conflicting access using locks
Lost Update	One update overwrites another	X-lock allows one writer at a time
Unrepeatable Read	Same read gives different values	Hold locks to prevent conflicting updates

Key point to remember:
Dirty Read → Uncommitted Data
Lost Update → Overwritten Update
Unrepeatable Read → Different Values on Re-read
2PL → Controls Access Using Locks

Q6. Deadlock in DBMS: Detection and Prevention

Interviewer Question:

“What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.”

Interview-Style Answer

A **deadlock** occurs when two or more transactions wait indefinitely for each other to release resources.

Example

Suppose:

T1 holds Lock A and waits for Lock B
T2 holds Lock B and waits for Lock A

Then:

T1 → waits for T2
↑ ↓
└──────────┘ T2 ─────────┘

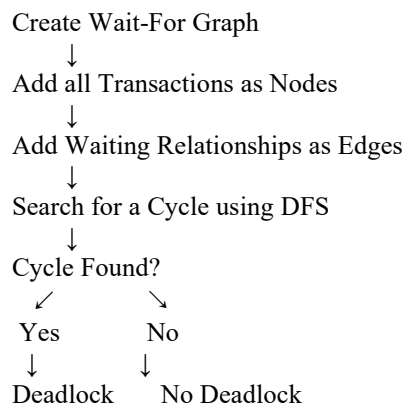
Neither transaction can continue, so a **deadlock occurs**.

1. Deadlock Detection Algorithm

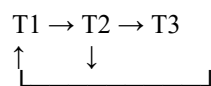
Deadlock can be detected using a **Wait-For Graph (WFG)**.

- Each node represents a transaction.
- An edge $T1 \rightarrow T2$ means T1 is waiting for T2.
- If a **cycle** exists, a deadlock is detected.

Algorithm Steps



For example:



Since there is a cycle, the system detects a deadlock.

2. Deadlock Prevention Strategies

Strategy 1: Resource Ordering

Resources are requested in a fixed order.

For example:

Transaction must request:
 $A \rightarrow B \rightarrow C$

A transaction cannot request B first and later request A. This prevents **circular wait**, one of the conditions required for deadlock.

Strategy 2: Wait-Die Scheme

This strategy uses transaction timestamps.

- If an **older transaction** requests a resource held by a younger transaction, it can **wait**.
- If a **younger transaction** requests a resource held by an older transaction, it is **aborted and restarted later**.

Older Transaction $\rightarrow$ Can WAIT
Younger Transaction $\rightarrow$ ABORT / Restart

This helps prevent cyclic waiting.

Another Strategy: Wound-Wait

In this method:

- An **older transaction** can force a younger transaction to abort.
- A **younger transaction** must wait if the resource is held by an older transaction.

Key point to remember:

Deadlock → Circular Waiting

Detection → Wait-For Graph + Cycle Detection

Prevention → Resource Ordering / Wait-Die / Wound-Wait

Q7. Immediate Update vs Deferred Update Recovery

Interviewer Question:

“Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?”

Interview-Style Answer

Both **Immediate Update** and **Deferred Update** are database recovery techniques used to maintain consistency after a system failure.

The main difference is **when the database is updated**.

1. Immediate Update

In the Immediate Update technique, database changes can be written **before the transaction commits**.

Therefore, if a system crashes:

- **Committed transactions** may require **REDO**.
- **Uncommitted transactions** require **UNDO**.

Example

T1: START

T1: Update A from 100 → 200

T1: COMMIT

If a crash occurs, the system uses the log to ensure T1's committed update is correctly reflected.

For an incomplete transaction:

T2: START
T2: Update B from 500 → 800
SYSTEM CRASH

Since T2 did not commit:

UNDO → Restore B = 500

Recovery Process

System Crash
↓
Read Log
↓
Committed Transactions → REDO
Uncommitted Transactions → UNDO

2. Deferred Update

In Deferred Update, database changes are **not written to the actual database until the transaction commits**.

Therefore, if a transaction fails before committing, its changes have not yet been applied to the database.

So:

- **Committed transactions** may require **REDO**.
- **Uncommitted transactions** generally do **not require UNDO**.

Example

T1: START
T1: Update A from 100 → 200
T1: COMMIT

After commit, the update is applied.

If:

T2: START
T2: Update B from 500 → 800
SYSTEM CRASH

Before T2 commits, its changes are not applied to the database. Therefore:

No UNDO required

Recovery Process

System Crash
↓
Read Log

↓
Find Committed Transactions
↓
REDO Required Updates
↓
Database Recovered

3. Comparison

Immediate Update	Deferred Update
Updates may reach database before commit	Updates are applied after commit
Requires UNDO and REDO	Mainly requires REDO
Handles uncommitted changes in database	Uncommitted changes are not written to database
Recovery is more complex	Recovery is simpler
Suitable when updates must be processed immediately	Suitable when delaying database updates is acceptable

4. When Is Each Preferred?

Immediate Update is preferred when:

- The system requires updates to be processed without waiting until final completion.
- High transaction activity needs flexible update handling.
- The DBMS uses logging and can perform both UNDO and REDO recovery.

Deferred Update is preferred when:

- A simpler recovery process is required.
- It is important to avoid uncommitted changes reaching the database.
- Updates can safely wait until the transaction commits.

Key point to remember:

Immediate Update → Before Commit → UNDO + REDO

Deferred Update → After Commit → Mainly REDO

Q8. Shadow Paging: Advantages and Limitations

Interviewer Question:

“Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?”

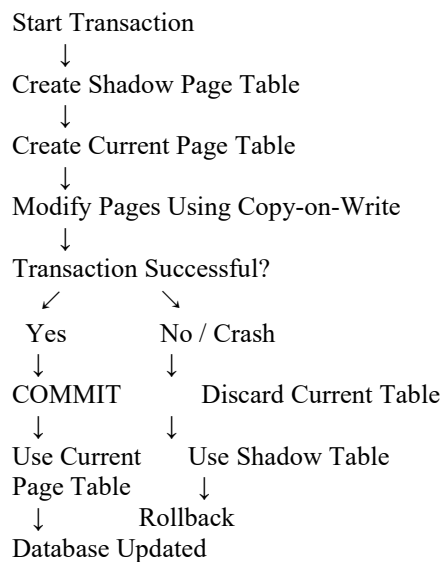
Interview-Style Answer

Shadow Paging is a recovery technique in which the database maintains two page tables:

- **Shadow Page Table** – Contains the original, stable version of the database.
- **Current Page Table** – Contains the pages modified by the current transaction.

The shadow page table is not changed during the transaction.

Working



Example

Suppose page P1 contains:

Balance = 10,000

During a transaction, instead of modifying the original page directly, a new copy is created:

Shadow Table → Original P1 → Balance = 10,000

Current Table → New P1 → Balance = 8,000

If the transaction commits, the **current page table becomes the new database state**. If a crash occurs before commit, the system uses the unchanged **shadow page table** to restore the original state.

Advantages of Shadow Paging

1. Simple Recovery

Recovery is relatively simple because the database can return to the shadow page table after a failure.

2. No UNDO Required for Uncommitted Changes

Uncommitted changes are stored separately from the original pages, so rollback can be performed by discarding the current page table.

3. Fast Rollback

Failure
↓
Discard Current Page Table
↓
Use Shadow Page Table

This can make recovery straightforward.

Limitations Compared to Log-Based Recovery

1. Page Table Management

Maintaining separate shadow and current page tables can create overhead.

2. Copy-on-Write Overhead

Every modified page may require creating a new page instead of directly updating the old page.

3. Fragmentation

Repeated creation of new pages can lead to scattered pages and fragmentation.

4. Less Flexible for Large Transactions

For complex or large transactions, managing many copied pages may become inefficient.

Comparison with Log-Based Recovery

Shadow Paging	Log-Based Recovery
Uses Shadow and Current Page Tables	Uses transaction logs
Keeps original pages unchanged	Records database changes in logs
Recovery can use the shadow table directly	Recovery uses log processing
Simple rollback for failures	Can use UNDO and REDO operations
Copy-on-write may create overhead	Log processing can be more complex
Can suffer from page fragmentation	More suitable for complex recovery management

Key point to remember:

Shadow Paging → Original Copy + Updated Copy

Commit → Use Current Table

Crash → Use Shadow Table

Q9. Serializability Schedule and Precedence Graph

Interviewer Question:

“What is a serializability schedule? How do you test for conflict serializability using a precedence graph?”

Interview-Style Answer

A **schedule** is the order in which operations of multiple transactions are executed.

A schedule is **serializable** if its result is equivalent to executing the transactions one after another in some serial order.

For example:

Serial Schedule:

T1 completes → Then T2 completes

A concurrent schedule may interleave operations:

T1: Read(A)

T2: Read(A)

T1: Write(A)

T2: Write(B)

If this concurrent execution produces the same result as some serial order, it is considered **serializable**.

Conflict Serializability

A schedule is **conflict serializable** if it can be transformed into an equivalent serial schedule by changing the order of **non-conflicting operations**.

Conflicts occur when two operations:

- Belong to different transactions.
- Access the same data item.
- At least one operation is a **WRITE**.

Examples:

Read-Write → Conflict

Write-Read → Conflict

Write-Write → Conflict

Read-Read → No Conflict

Testing Using a Precedence Graph

A **precedence graph**, also called a **serialization graph**, is used to test conflict serializability.

Step 1: Create Nodes

Create one node for each transaction:

T1 T2 T3

Step 2: Find Conflicting Operations

Suppose we have:

T1: Write(A)

T2: Read(A)

Since both transactions access A and one operation is a write, there is a conflict.

Because T1 performs the conflicting operation first:

T1 → T2

Add a directed edge from T1 to T2.

Step 3: Check All Conflicts

Repeat this for all conflicting operations and construct the complete precedence graph.

Step 4: Check for a Cycle

$T1 \rightarrow T2 \rightarrow T3$

If there is **no cycle**, the schedule is conflict serializable.

The possible serial order is obtained by performing a **topological sort**.

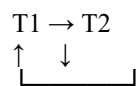
For example:

$T1 \rightarrow T2 \rightarrow T3$

Serial order:

$T1 \rightarrow T2 \rightarrow T3$

Example of a Non-Serializable Schedule



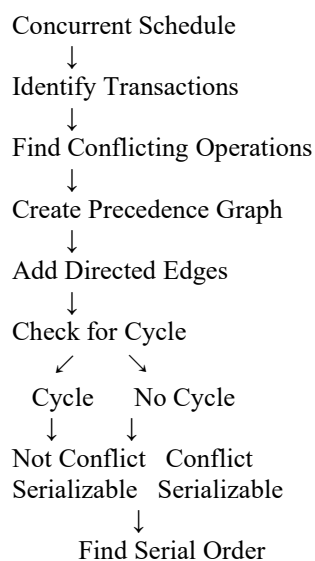
This graph contains a cycle:

$T1 \rightarrow T2 \rightarrow T1$

Therefore:

Cycle Found
↓
Not Conflict Serializable

Complete Testing Flow



Key point to remember:

Precedence Graph → No Cycle = Conflict Serializable

Precedence Graph → Cycle = Not Conflict Serializable

Q10. Variations of Two-Phase Locking (2PL)

Interviewer Question:

“Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?”

Interview-Style Answer

Two-Phase Locking (2PL) is a concurrency control protocol used to ensure correct transaction execution. It controls access to data using locks and helps achieve serializability.

There are three main variations:

1. Basic 2PL

Basic 2PL has two phases:

Growing Phase
Acquire Locks Only
↓
Shrinking Phase
Release Locks Only

- During the **growing phase**, a transaction can acquire locks.
- During the **shrinking phase**, it can release locks.
- Once a transaction releases its first lock, it cannot acquire any new locks.

Advantage: Ensures conflict serializability.

Limitation: Deadlocks can still occur, and releasing locks before commit may allow cascading rollback.

2. Strict 2PL

In **Strict 2PL**, exclusive (**X**) locks are held until the transaction **commits or aborts**.

Acquire X-Lock
↓
Perform Operations
↓
COMMIT / ABORT
↓
Release X-Lock

This prevents other transactions from reading or modifying data written by an uncommitted transaction.

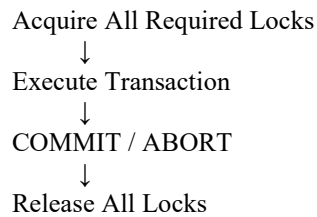
Advantage: Prevents dirty reads and cascading rollbacks.

3. Rigorous 2PL

In **Rigorous 2PL**, **all locks**, including:

- Shared locks (**S-locks**)
- Exclusive locks (**X-locks**)

are held until the transaction commits or aborts.



This is stricter than Strict 2PL because even shared locks are not released before the transaction finishes.

Advantage: Provides strong isolation and simple serial execution ordering.

Limitation: Can reduce concurrency because locks are held for a longer time.

Comparison

Feature	Basic 2PL	Strict 2PL	Rigorous 2PL
Two phases	Yes	Yes	Yes
Prevents dirty reads	Not always	Yes	Yes
X-lock held until commit/abort	No	Yes	Yes
S-lock held until commit/abort	No	No	Yes
Concurrency	Higher	Good balance	Lower
Cascading rollback	Possible	Prevented	Prevented

Which Is Most Commonly Used?

Strict 2PL is commonly used because it provides a good balance between **data consistency, recovery, and concurrency**.

It prevents transactions from accessing uncommitted updates, which avoids **dirty reads and cascading rollbacks**, while usually allowing more concurrency than Rigorous 2PL.

Key point to remember:

Basic 2PL → Acquire, then Release

Strict 2PL → Hold X-locks until Commit

Rigorous 2PL → Hold All Locks until Commit

Most commonly used → Strict 2PL